1 Segment Tree

1.1 Normal

```
// Indexada em 0, [a, b]
class Segtree{
  private:
    ll N;
    vector<ll> ts;
  public:
    // o 0 eh elemento neutro da soma
    Segtree(size_t n) : N(n), ts(4*n, 0) {}
    ll query(ll ql, ll qr){
      return query_range(ql, qr, 0, N-1, 1);
    }
    void update(ll i, ll v){
      return update_range(i, v, 0, N-1, 1);
    }
  private:
    ll query_range(ll ql, ll qr, ll l,ll r, ll no){
      if(l >= ql && r <= qr) return ts[no];
      if(l > qr || r < ql) return 0;
      ll mid = (l+r)/2;
      ll esq = query_range(ql, qr, l, mid, 2*no);
      ll dir = query_range(ql, qr, mid+1, r, 2*no+1);
      return merge(esq, dir);
    }
    void update_range(ll i, ll v, ll l, ll r, ll no){
      if(l==r){
        ts[no] = v;
        return;
      }
      ll mid = (l+r)/2;
      if(i<=mid) update_range(i, v, l, mid, 2*no);
      else update_range(i, v, mid+1, r, 2*no+1);
      ts[no] = merge(ts[2*no], ts[2*no+1]);
    }
    ll merge(ll a, ll b){
      return a+b;
    }
};
```

1.2 Lazy Propagation

```
template<typename T> class SegmentTree {
    private:
        int N;
        vector<T> ns, lazy;
```

```
5       public:
6           SegmentTree(const vector<T>& xs)
7               : N(xs.size()), ns(4*N, 0), lazy(4*N, 0) {
8               for(size_t i = 0; i < xs.size(); i++)
9                   update(i, i, xs[i]);
10          }
11          void update(int a, int b, T value) {
12              update(1, 0, N-1, a, b, value);
13          }
14      private:
15          void update(int node, int L, int R, int a, int b, T
                value) {
16              if(lazy[node]) {
17                  ns[node] += (R - L + 1) * lazy[node];
18                  if(L < R) {
19                      lazy[2*node] += lazy[node];
20                      lazy[2*node + 1] += lazy[node];
21                  }
22                  lazy[node] = 0;
23              }
24              if(a > R || b < L)
25                  return;
26              if(a <= L && b >= R) {
27                  ns[node] += (R - L + 1) * value;
28                  if(L < R) {
29                      lazy[2*node] += value;
30                      lazy[2*node + 1] += value;
31                  }
32                  return;
33              }
34              update(2*node, L, (L + R)/2, a, b, value);
35              update(2*node + 1, (L + R)/2 + 1, R, a, b, value
                    );
36              ns[node] = ns[2*node] + ns[2*node + 1];
37          }
38      public:
39          T RSQ(int a, int b) {
40              return RSQ(1, 0, N - 1, a, b);
41          }
42      private:
43          T RSQ(int node, int L, int R, int a, int b) {
44              if(lazy[node]) {
45                  ns[node] += (R - L + 1) * lazy[node];
46                  if(L < R) {
47                      lazy[2*node] += lazy[node];
48                      lazy[2*node + 1] += lazy[node];
49                  }
50                  lazy[node] = 0;
51              }
52              if(a > R || b < L)
```

```
                return 0;
            if(a <= L and b >= R)
                return ns[node];
            T x = RSQ(2*node, L, (L + R)/2, a, b);
            T y = RSQ(2*node + 1, (L + R)/2 + 1, R, a, b);
            return x + y;
        }
};
```

2 Algoritmo de Kahn

```
// Na construcao da entrada, faca in[x].insert(y), out[y].
    insert(x)
vll kahn(ll n, set<ll> in[], set<ll> out[]) {
    vll o;
    priority_queue<ll, vll, greater<ll>> q;
    rep(i, 0, n) {
        if(in[i].empty())
            q.push(i);
    }
    while(!q.empty()) {
        ll u = q.top(); q.pop();
        for(auto e: out[u]) {
            in[e].erase(u);
            if(in[e].empty())
                q.push(e);
        }
        o.push_back(u);
    }
    return (o.size()==n ? o : vll()); // Retorna vazio, caso
         haja ciclo
}
```

3 Verificar se o grafo é bipartido

```
// as cores sao 0 e 1
bool isBipartite(ll s, ll n, const vvll &adj) {
    queue<ll> q;
    q.push(s);
    vll color(n, -1); color[s]=0;
    bool flag = true;
    while (!q.empty())
    {
        for(auto nex: adj[q.front()]) {
            if(color[nex] == -1) {
```

```
                color[nex] = 1-(color[q.front()]);
                q.push(nex);
            }
            else if(color[nex] == color[q.front()]) {
                flag = false;
                break;
            }
        }
        q.pop();
    }
    return flag;
}
```

4 Dijkstra

```
/**
 *  @param  g  Graph (w, v).
 *  @param  s  Starting vertex.
 *  @return    Vectors with smallest distances from every
    vertex to s and the paths.
 *  Na construcao do grafo, nos pares, primeiro vem o peso
 *  e depois o vertice
*/
pair<vll, vll> dijkstra(const vvpll& g, ll s) {
    vll ds(g.size(), LLONG_MAX), pre = ds;
    priority_queue<pll, vpll, greater<>> pq;
    ds[s] = 0, pq.emplace(ds[s], s);
    while (!pq.empty()) {
        auto [t, u] = pq.top(); pq.pop();
        if (t > ds[u]) continue;
        for (auto [w, v] : g[u])
            if (t + w < ds[v]) { // relaxamento
                ds[v] = t + w, pre[v] = u;
                pq.emplace(ds[v], v);
            }
    }
    return { ds, pre };
}
vll get_path(const vll& pre, ll u) {
    vll p;
    while (u != LLONG_MAX) {
        p.eb(u), u = pre[u];
    }
    reverse(all(p));
    return p;
}
```

5 Kruskal (MST)

```
#define tll   tuple<ll, ll, ll>
#define vtll  vector<tll>
struct DSU {
    vll parent, size;
    DSU(ll sz) : parent(sz), size(sz, 1) { iota(all(parent),
        0); }

    ll find(ll x) {
        assert(0 <= x && x < parent.size());
        return parent[x] == x ? x : parent[x] = find(parent[
            x]);
    }
    void merge(ll x, ll y) {
        ll a = find(x), b = find(y);
        if (size[a] > size[b]) swap(a, b);
        parent[a] = b;
        if (a != b) size[b] += size[a], size[a] = 0;
    }
    bool same(ll x, ll y) { return find(x) == find(y); }
};
vtll kruskal(vtll& edges, ll n) {
    DSU dsu(n);
    vtll mst;
    sort(all(edges));  // change order if want maximum
    for (auto [w, u, v] : edges) if (!dsu.same(u, v)) {
        dsu.merge(u, v);
        mst.eb(w, u, v);
    }
    return mst;
}
```